# Cola da defesa - HomeLive 2050

Java 21 + Spring Boot 4 + PostgreSQL. Caminho de uma requisição: front (HTML/JS) → controller → service → repository → banco. As **regras ficam nas entidades**; o service só orquestra.

## Conceito → onde está no código

| Conceito | Onde | Em uma frase |
|---|---|---|
| **Herança e polimorfismo** | `model/Usuario` (abstrata, `@Inheritance(JOINED)`) → `Morador`, `Sindico` | Cada perfil implementa `getTipoUsuario()`; o Spring Security usa isso como permissão (`ROLE_`). No banco: `usuarios` guarda o comum, `moradores` e `sindicos` usam o mesmo id. |
| **Encapsulamento** | `Despesa.aprovar/rejeitar`, `AgendamentoEspaco.cancelar`, `TarefaDomestica.concluir` | Entidades **sem setters**: o estado só muda por métodos que recusam transição inválida (despesa já avaliada não é avaliada de novo). O construtor de `AgendamentoEspaco` recusa reserva no passado. `EncapsulamentoEntidadesTest` falha se alguém criar um setter público. |
| **Strategy** | `strategy/CalculoRateioStrategy` (Igualitário, Estadia, Consumo) e `RodizioStrategy` (Sequencial, Menor Pontuação, Menor Carga) | Troca o algoritmo sem `if/else` gigante. A `RateioStrategyFactory` escolhe a estratégia pelo enum `TipoRateio`. Critério novo = classe nova. |
| **Template Method** | `RateioProporcionalStrategy.calcular()` (`final`) e `RodizioBaseStrategy` | O esqueleto (validar, somar pesos, repartir, fechar os centavos no último morador) fica na classe abstrata; as filhas só dizem o `peso()` de cada morador. |
| **Observer** | `event/` (3 eventos) e `PenalidadeService` com `@EventListener` | `TarefaService` e `AgendamentoService` só publicam (`publishEvent`); quem aplica a penalidade é o `PenalidadeService`. Tarefa e reserva não sabem que penalidades existem. |
| **Strategy no enum** | `ModoCalculoPenalidade`, `Periodicidade` | Cada valor do enum sabe calcular: FIXA/PROGRESSIVA (pontos) e DIARIA/SEMANAL/MENSAL (próxima data). |
| **Estruturas de dados** | fábricas, `ContextoRateio`, `ContextoRodizio`, `TarefaRecorrenteService` | `EnumMap<Tipo, Strategy>` (busca direta pela chave); `Map<Long,BigDecimal>` (kWh por morador) e `Map<Long,Long>` (tarefas abertas por morador, via `groupingBy`); `List` ordenada com `Comparator` no rodízio circular; `Optional` p/ ausência; `BigDecimal` p/ dinheiro. |
| **Conflito de horário** | `AgendamentoEspacoRepository.findConflitos` | Há conflito se `inicioNovo < fimExistente E fimNovo > inicioExistente` (só reservas CONFIRMADAS do mesmo espaço). Resposta 409. |
| **Validação** | DTOs (`@NotBlank` `@Email` `@Positive` `@Size`), `GlobalExceptionHandler`, front | O back-end recusa dado inválido com 400 em JSON; o front também (`type=number`, `required`, telefone só aceita dígitos e símbolos). Regra de negócio quebrada: `RegraNegocioException`. |
| **Autenticação e perfis** | `security/SecurityConfig`, `TokenService`, `TokenAuthenticationFilter`, `UsuarioAutenticado` | Senha com **BCrypt**. `POST /api/auth/login` devolve um token JWT (HS256; 30 dias com "continuar conectado"). Permissões por perfil numa matriz só, em `SecurityConfig`; regra de "dono" em `UsuarioAutenticado`. |
| **Automação e testes** | `AgendadorConfig` (`@Scheduled`, 06:00); `src/test` | Todo dia às 06:00: marca tarefas atrasadas (dispara penalidades) e gera as tarefas do rodízio. 251 testes (unidade, integração e segurança). |

## Perguntas prováveis

**Por que herança JOINED?**
Evita colunas vazias: o comum fica em `usuarios` e cada perfil guarda só o que é seu.

**Por que o front não manda "quem sou eu"?**
A identidade vem do token, não do corpo da requisição. Assim um morador não consegue se passar por síndico.

**Por que token e não guardar a senha?**
A senha não fica no navegador. O token é assinado e expira; se mudar um byte, a assinatura não confere.

**Como a senha é guardada?**
Só o hash BCrypt, nunca o texto.

**Como o rateio fecha os centavos?**
O último morador recebe a diferença, então a soma sempre dá o valor total.

**O que acontece com tarefa atrasada?**
A rotina das 06:00 marca ATRASADA e publica um evento; o `PenalidadeService` aplica a regra ativa (FIXA ou PROGRESSIVA).

**Como entra um morador novo?**
O 1º cadastro cria a residência e vira síndico; os seguintes entram com o código de convite (`HL-XXXXXX`) e viram moradores.

**Por que uma residência só?**
Decisão de escopo: várias casas exigiriam filtrar todos os dados por casa. Ficou como próxima versão.

**Limitações (assumir como decisão):** uma residência por instalação; sem recuperação de senha por e-mail (o síndico gera uma senha temporária); segredo do token e senha do banco com valor de desenvolvimento; sem limite de tentativas de login; sem HTTPS; PWA testado só no computador.

**Rodar:** `./mvnw spring-boot:run -Dspring-boot.run.profiles=postgres` → `http://localhost:8080`. Plano B sem Postgres: tirar o `-D...` (banco em memória). Do zero: `DEMO_DATA=false`. Contas de exemplo: `helena.vance@homelive2050.io` (síndica) e `lucas.silva@homelive2050.io` (morador), senha `homelive123`.